Zhengchun Shang and Wenlan Wei. Evolving security in llms: A study of jailbreak attacks and defenses. *arXiv preprint arXiv:2504.02080*, 2025.

Jiuding Sun, Jing Huang, Sidharth Baskaran, Karel D'Oosterlinck, Christopher Potts, Michael Sklar, and Atticus Geiger. HyperDAS: Towards automating mechanistic interpretability with hypernetworks. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2503.10894.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/scaling-monosemanticity.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *arXiv preprint arXiv:2305.04388*, 2023.

Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations*, 2023.

Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems*, 36:80079–80110, 2023.

# A Appendix

## A.1 Training Details

During pretraining, we prepend all web text passages with the following `INSTRUCT_PREFIX`:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n\n
Cutting Knowledge Date: December 2023\nToday Date: 26 Jul 2024\n\n
<|eot_id|><|start_header_id|>user<|end_header_id|>\n\n
```

For additional training hyperparameters, we use a LoRA alpha value of 32, a dropout of 0.05, a learning rate of $10^{-4}$, an effective batch size of 128, a weight decay of 0.01, and no learning rate warmup.

For the auxiliary loss, we use $\epsilon_{\text{aux}} = 10^{-4}$, $k_{\text{aux}} = 500$ and find that the training is not very sensitive to changes in these parameters.

## A.2 Additional Results on Dead Concepts

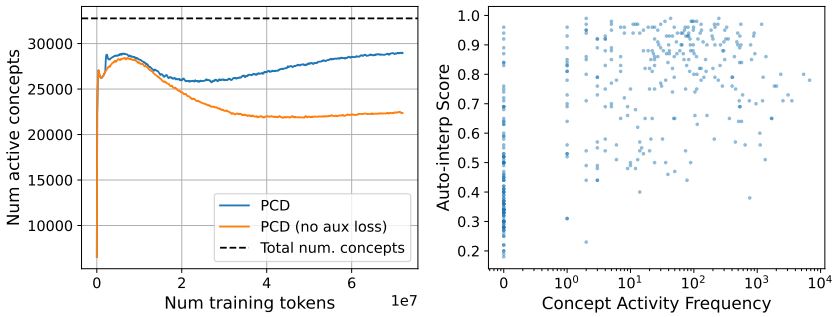



Figure 13: **Left:** Without the auxiliary loss, the number of active concepts drops substantially during training; the auxiliary loss brings dead concepts back alive. **Right:** Higher-frequency concepts generally have higher auto-interp scores; many dead concepts score poorly.

Figure 13 shows how our auxiliary loss improves concept activity in a 72M training run, and how many inactive concepts have poor interpretability.

We also tried two other strategies for reducing the number of inactive concepts over the course of training.

**Normalizing Encoder Directions**. Since the top-activating concepts for an activation $\mathbf{a}$ are selected by computing $\text{TopK}(\mathbf{W}_{\text{enc}}(\mathbf{a}) + \mathbf{b}_{\text{enc}})$, there may be a bias towards concepts $\mathbf{W}_{\text{enc},i}$ with larger norm. We considered normalizing all concepts before computing the TopK.

**Annealing**. Other work in the SAE literature (He et al., 2024) found that it was possible to avoid inactive concepts by gradually annealing the number of active concepts from $d$ to $k$ over the first $10\%$ of training.

We did not find either strategy effective for reducing the number of inactive concepts. We also did not directly try the auxiliary loss in (Gao et al., 2024) because adapting it to our setting would require twice as many decoder forward and backward passes.

## A.3 SAE Baselines

For the regular SAE baselines, we trained standard TopK SAEs following (Gao et al., 2024). That is, we compute the reconstruction $\mathbf{a}' = W_{\text{dec}}(\text{TopK}(\mathbf{W}_{\text{enc}}(\mathbf{a} - \mathbf{b}_{\text{enc}})))$ and the training objective is to minimize $\|\mathbf{a} - \mathbf{a}'\|_2^2$.

For the KL SAE baseline on the $n_{\text{middle}}$ tokens, we first run a regular forward pass of the subject model $\mathcal{S}$ with the regular layer $\ell_{\text{read}}$ residual stream activations $\mathbf{a}$ to obtain probabilities $p$ over the

$n_{\text{middle}}$ tokens. Then, we compute the reconstruction $\mathbf{a}'$ as above, and run a second forward pass of $\mathcal{S}$ with $\mathbf{a}'$ patched into the layer $\ell_{\text{read}}$ residual stream activations at $n_{\text{middle}}$ tokens to obtain a probability vector $p'$. The training objective is then $\text{KL}(p, p')$.

For the KL SAE baseline on the $n_{\text{suffix}}$ tokens, we perform the same procedure as in the previous paragraph, except $p$ and $p'$ are computed over the $n_{\text{suffix}}$ tokens.

### A.4 Effect of Varying Hyperparameters

Unless otherwise stated, all experiments in this section were conducted training PCD on 18M tokens.

#### A.4.1 Training Objective

We initially expected to see improvements from changing the pretraining objective to incorporate the subject model's probabilities. Specifically, we tried replacing the next-token prediction loss

$$\mathcal{L}_{\text{next-token}} = -\sum_{t=1}^{n_{\text{suffix}}} \log p_{\mathcal{D}}(s^{(t)} \mid s^{(1:t-1)}, \mathcal{E}(\mathbf{a}^{(1:n_{\text{middle}})})). \tag{4}$$

with a loss to encourage matching the subject model's predictions

$$\mathcal{L}_{\text{KL}} = \sum_{t=1}^{n_{\text{suffix}}} D_{\text{KL}}\big(p_{\mathcal{S}}(\cdot \mid s^{(1:t-1)}) \parallel p_{\mathcal{D}}(\cdot \mid s^{(1:t-1)}, \mathcal{E}(a^{(1:n_{\text{middle}})}))\big). \tag{5}$$

However, we found that this change seemed to cause an earlier plateau in encoder interpretability metrics.

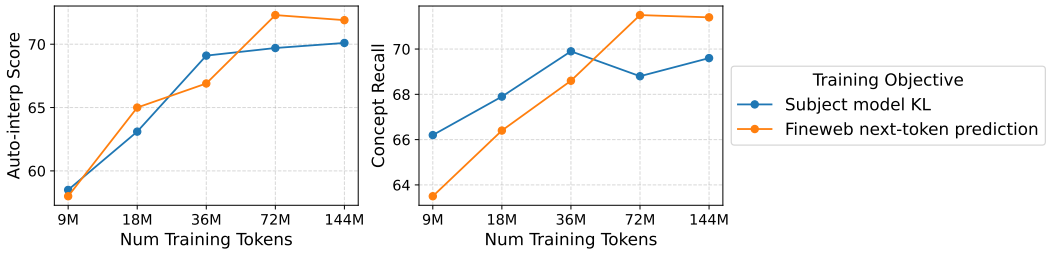



Figure 14: Comparison of regular and KL-based training objectives.

#### A.4.2 LoRA Rank

We did not observe a clear trend when changing the PCD decoder's LoRA rank, as seen in Figure 15.

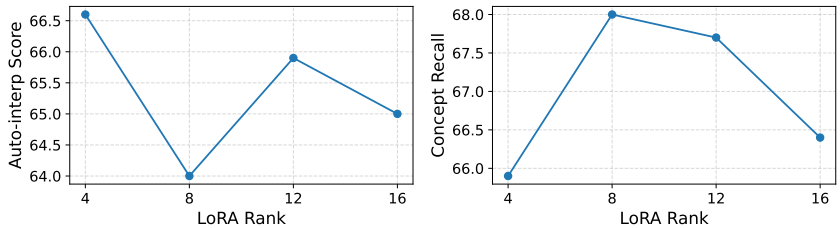



Figure 15: The effect of LoRA rank on encoder concept interpretability was ambiguous.

#### A.4.3 Number of Active Concepts

Figure 16 shows there may be small gains from increasing the number of active concepts in the bottleneck, though this comes at the cost of degraded human interpretability.

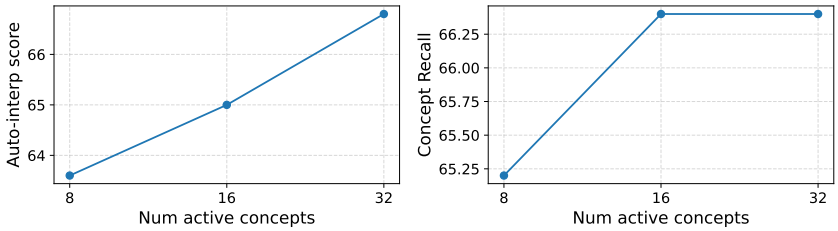



Figure 16: There may be slight interpretability gains from increasing the number of active concepts in the encoder.

### A.5 Additional Finetuning Results

We present full results of evaluating intermediate checkpoints from finetuning PCDs and baselines on SynthSys.

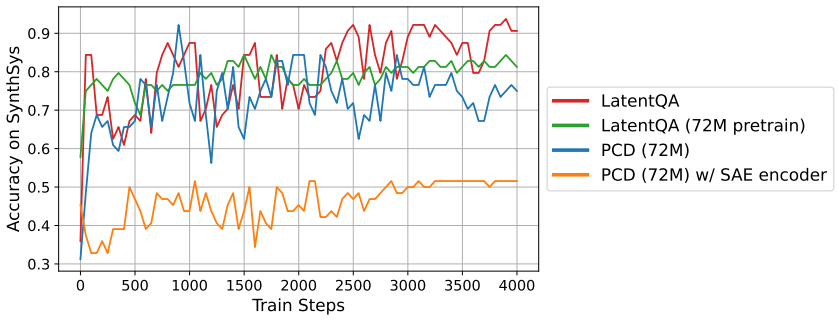



Figure 17: We evaluate accuracy on held-out attributes during finetuning on SynthSys for various baselines as well as PCDs pretrained on 72M FineWeb tokens

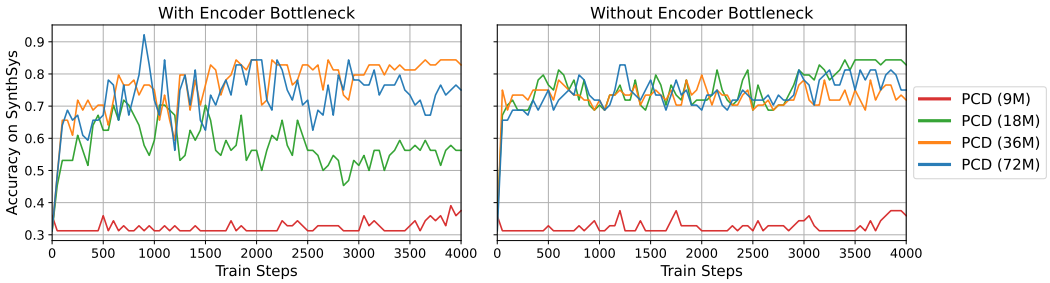



Figure 18: We evaluate accuracy on held-out attributes during finetuning on SynthSys for PCD models pretrained on {9M, 18M, 36M, 72M} tokens. While finetuning is done with the bottleneck, we evaluate both with the bottleneck (left) and without the bottleneck (right).